

TP2

Typescript

Ejercicio 1

Considera el lenguaje TypeScript acotado al paradigma de programación estructurada y analízalo en términos de [los cuatro componentes de un paradigma](#) mencionados por Kuhn.

1. Generalización simbólica: ¿Cuáles son las reglas escritas del lenguaje?

Las reglas escritas del lenguaje analizan la sintaxis, reglas gramaticales y formalismos del lenguaje que los desarrolladores aceptan y el compilador exige cumplir.

Por eso analizamos los siguientes puntos

- **Estructura de control Imperativa y Secuencial:**

Uso de bloques delimitados por llaves {} y punto y coma opcional o explícito como separador de sentencias

Condicionales (if / else, switch) y bucles explícitos (for, while, do-while)

- **Declaración formal y análisis de tipos operacional:**

Reglas de anotación de tipos explícitas mediante:

type (ej. let contador: number = 0;).

Firmas formales para funciones puras/procedimentales:

Function suma(a: number, b:number): number { return a+b}.

Definición de contratos de datos mediante interfaces o alias de tipos estructurados(type / interface) para tipar parámetros y valores de retorno sin recurrir a clases.

- **Manejo estricto de alcance (Scoping);**

Bloque de alcance léxico definido por let y const, prohibieron el reuso no deseado de variables (evitando el hoisting tradicional de var).

2. Creencias de los profesionales: ¿Qué características particulares del lenguaje se cree que sean "mejores" que en otros lenguajes?

- **El tipado estático gradual y expresivo previene errores en tiempo de compilación:**

Se cree firmemente que detectar incompatibilidades de tipos mediante el transpilador (build time) es drásticamente más eficiente y menos propenso a errores que descubrirlos en ejecución (runtime), superando la fragilidad sintáctica de JavaScript tradicional.

- **Transparencia y documentación auto-contenida:**

La comunidad considera que las firmas de función tipadas y los tipos de datos personalizados sirven como documentación viva. Se cree que un código estructurado en Typescript describe mejor la intención del programador que los comentarios en código no tipado.

- **Seguridad sin comprometer flexibilidad (Tipado Estructural):**

A diferencia de lenguajes con tipado nominal estricto (como C++ o Java), en TypeScript predomina la creencia de que si dos estructuras tienen la misma forma, son compatibles ("Duck Typing" formalizado). Esto permite mantener programas estructurados limpios y desacoplados sin sobrecarga de herencia de jerarquías complejas.

- **Inmutabilidad y Predictibilidad mediante const :**

Existe un consenso profesional que favorece el uso sistemático de const frente a let para prevenir mutaciones colaterales en el flujo de ejecución procedural